

LAPORAN TUGAS KRIPTOGRAFI RSA
Implementation of RSA for E-Certificate Integrity and
Verification



Kelompok 17

5025231246	Naswan Nashir Ramadhan
5025241164	Alfianz Risqia Ilahi Loven Kary
5027241087	Az Zahrra Tasya Adelia

INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026

Daftar Isi

1.	Pendahuluan	3
1.1.	Latar Belakang	3
1.2.	Tujuan	3
2.	Dasar Teori	4
2.1.	RSA.....	4
2.2.	Key Generation	4
2.3.	RSA Signing	5
2.4.	RSA Verification	5
2.5.	Hash Function (SHA-256)	6
2.6.	Integritas Dokumen.....	6
3.	Perancangan dan Implementasi Sistem	7
3.1.	Gambaran Umum Sistem	7
3.2.	Proses Key Generation.....	8
3.3.	Implementasi Modular Exponentiation	10
3.4.	Proses Signing.....	12
3.5.	Pembentukan RSA Token	13
3.6.	Penyimpanan Token pada Metadata.....	14
3.7.	Proses Verification	15
3.8.	Pemeriksaan Expiry	17
3.9.	Antarmuka Sistem.....	19
4.	Analisis Hasil	22
4.1.	Analisis Verifikasi Integritas	22
4.2.	Analisis Hasil Pengujian	22
4.3.	Keterbatasan Sistem	23
5.	Kesimpulan.....	23
6.	Lampiran	24
6.1.	Screenshot bagian app.py.....	24
6.2.	Screenshot bagian cert_service.py	24
6.3.	Screenshot bagian requirement.txt	24
6.4.	Screenshot bagian rsa_core.py	25
6.5.	Screenshot bagian cek_metadata.py	25
6.6.	Link github repository.....	25

1. Pendahuluan

1.1. Latar Belakang

Sekarang sertifikat bukan hanya digunakan dalam bentuk fisik, tetapi juga bentuk digital atau e-certificate. Alasannya karena e-certificate lebih mudah dalam proses pembuatan, penyimpanan, dan pengirimannya. Namun, e-certificate juga masih memiliki kekurangan karena isinya yang dapat mudah diubah. Misalnya, data seperti nama, nilai, atau tanggal mudah sekali untuk dimanipulasi. Oleh karena itu, dibutuhkan cara untuk tahu apakah sertifikat tersebut masih sama seperti saat pertama kali diterbitkan.

Salah satu cara yang dapat digunakan adalah digital signature. Digital signature ini digunakan untuk memberi tanda pada dokumen menggunakan private key dari penerbit. Lalu, public key dapat digunakan untuk melakukan pengecekan terhadap tanda tangan tersebut. Karena proses ini menggunakan hash, jika isi dokumen berubah, nilai hash juga akan berubah. Hal ini dapat membuat kita lebih mudah untuk mengetahui apakah dokumen telah diubah atau tidak.

RSA adalah salah satu algoritma kriptografi yang bisa diimplementasikan untuk proses digital signature. Pada sistem ini, RSA digunakan untuk melakukan signing dan verification pada sebuah e-certificate. Hash digunakan sebagai bagian dari data yang akan ditandatangani dengan private key. Hasil tanda tangan tersebut kemudian disimpan pada metadata file sertifikat.

Sistem yang dibuat akan menunjukkan proses RSA dari awal sampai akhir, mulai dari pembuatan public key dan private key, proses signing, sampai proses verification. Selain itu, sistem juga dapat digunakan untuk mencoba beberapa kondisi sertifikat, seperti sertifikat yang masih valid, sertifikat yang isinya sudah diubah, dan sertifikat yang sudah melewati tanggal berlaku.

1.2. Tujuan

Tujuan dari pembuatan sistem ini adalah:

- Mengimplementasikan algoritma RSA secara manual untuk proses pembuatan kunci, signing, dan verification.

- Menerapkan digital signature pada e-certificate untuk membantu mengecek apakah isi sertifikat telah mengalami perubahan.
- Menggunakan hash sebagai bagian dari proses pengecekan integritas e-certificate.
- hasil tanda tangan digital pada metadata file sertifikat tanpa mengubah tampilan sertifikat.
- Membuat sistem yang dapat menunjukkan proses RSA dan melakukan pengujian terhadap beberapa kondisi sertifikat, seperti sertifikat valid, sertifikat yang telah dimodifikasi, dan sertifikat yang telah melewati masa berlaku.

2. Dasar Teori

2.1. RSA

RSA adalah salah satu algoritma kriptografi yang menggunakan dua buah kunci, yaitu public key dan private key. Dalam sistem ini fungsi public key adalah digunakan untuk melakukan verifikasi, sedangkan private key digunakan dalam proses signing.

RSA bekerja menggunakan operasi matematika pada bilangan yang cukup besar. Keamanan RSA ditentukan oleh kesulitannya dalam melakukan faktorisasi pada hasil perkalian dua bilangan prima yang besar. Dalam implementasi pada sistem ini, tahapan RSA ada tiga, yaitu pembuatan pasangan kunci, proses signing, dan verification.

Pada sistem ini, RSA digunakan untuk memberikan tanda tangan digital pada e-certificate. Data yang ingin ditandatangani diproses dulu menjadi hash, lalu digunakan sebagai bagian payload yang diproses menggunakan private key. Hasil proses tadi lalu disimoan sebagai RSA token pada metada sertifikat yang bisa diperiksa dengan public key

2.2. Key Generation

Key generation adalah proses pembuatan pasangan public key dan private key yang digunakan di RSA. Pada sistem ini, proses key generation dilakukan tanpa menggunakan library.

Proses ini dimulai dengan memilih dua buah bilangan prima, misal p dan q. Lalu, dua bilangan itu digunakan untuk menghitung nilai n dan $\phi(n)$. Rumus mencarinya adalah :

$$n = p \cdot q$$

$$\phi(n) = (p-1)(q-1)$$

Lalu suatu nilai (disini menggunakan e) dipilih selama syarat $1 < e < \phi(n)$ dipenuhi dan GCD e dengan $\phi(n)$ adalah 1. Nilai d lalu dicari sehingga memenuhi :

$$d \cdot e \equiv 1 \pmod{\phi(n)}$$

Setelah semua didapatkan, pasangan kunci digunakan dalam :

$$\text{Public Key} = (e, n)$$

$$\text{Private Key} = (d, n)$$

Pada sistem, proses diatas langsung dilakukan oleh program (Termasuk pengecekan bilangan prima, GCD, dan mencari nilai d).

2.3. RSA Signing

RSA signing adalah proses membuat tanda tangan digital dari data yang nantinya akan digunakan pada proses verification. Pada sistem ini, data dari hash dokumen digabungkan dengan informasi tanggal berlaku dan nantinya akan menjadi sebuah payload.

Payload itu lalu dibagi menjadi beberapa blok agar tiap blok lebih kecil nilainya dengan n pada RSA. Tiap blok lalu diproses dengan private key dengan rumus :

$$s = m^d \bmod n$$

Keterangan :

- m = nilai blok dari payload
- d = private exponent
- n = modulus RSA
- s = hasil signing (signature)

Hasil pada tiap blok digabung menjadi RSA token. Token itu lalu disimpan pada metadata file sertifikat yang nantinya digunakan pada proses verification.

2.4. RSA Verification

RSA verification adalah proses memeriksa tanda tangan yang sebelumnya digunakan menggunakan private key. Pada sistem, verification digunakan dengan menggunakan public key.

RSA token pada metadata sertifikat diambil lalu diproses menjadi beberapa blok lalu tiap blok dihitung menggunakan public key dengan rumus :

$$m = s^e \bmod n$$

Keterangan :

- s = signature dari hasil signing (pada proses RSA signing)
- e = public exponent
- n = modulus RSA
- m = nilai blok payload yang diperoleh kembali

Setelah semuanya diproses, hasilnya digabungkan kembali agar mendapatkan payload. Payload itu lalu dibandingkan dengan has dari dokumen. Jika hash sesuai dengan tanggal berlaku sertifikat masih aktif, sertifikat dinyatakan valid. Jika hash berbeda, berarti ada perubahan pada isi dokumen.

2.5. Hash Function (SHA-256)

Hash function adalah proses untuk mengubah suatu data menjadi nilai dengan panjang tertentu yang disebut hash. Nilai hash dapat digunakan untuk mewakili isi dari suatu data. Jika isi data berubah, nilai hash yang dihasilkan juga akan berubah.

Pada sistem ini digunakan SHA-256 untuk menghasilkan hash dari isi e-certificate. Hash tersebut kemudian digunakan sebagai bagian dari payload yang akan diproses dalam RSA signing. Saat dilakukan verification, sistem akan menghitung kembali hash dari e-certificate dan membandingkannya dengan hash yang terdapat pada payload hasil verification.

Jika kedua hash sama, maka isi dokumen yang diperiksa masih sesuai dengan isi dokumen saat dilakukan signing. Sebaliknya, jika hash berbeda, maka dapat diketahui bahwa isi dokumen telah mengalami perubahan.

2.6. Integritas Dokumen

Integritas dokumen digunakan untuk memastikan bahwa isi dokumen tidak mengalami perubahan setelah diterbitkan. Pada sistem ini, pengecekan integritas dilakukan dengan menggunakan hash dan RSA signature.

Saat sertifikat diterbitkan, sistem akan menghitung hash dari isi sertifikat. Hash tersebut kemudian dimasukkan ke dalam payload dan ditandatangani menggunakan private key. Hasil signing disimpan sebagai RSA token pada metadata sertifikat.

Saat sertifikat diverifikasi, sistem mengambil RSA token dari metadata dan melakukan verification menggunakan public key. Dari proses tersebut akan diperoleh kembali hash yang sebelumnya digunakan saat signing. Sistem kemudian menghitung hash dari sertifikat yang sedang diperiksa dan membandingkannya dengan hash tersebut.

Jika kedua hash sama, isi sertifikat dianggap masih sesuai dengan saat diterbitkan. Jika berbeda, berarti terdapat perubahan pada isi sertifikat. Selain itu, sistem juga memeriksa tanggal berlaku yang terdapat pada payload untuk menentukan apakah sertifikat masih berada dalam masa berlaku.

3. Perancangan dan Implementasi Sistem

3.1. Gambaran Umum Sistem

Sistem yang dibuat merupakan aplikasi untuk melakukan penerbitan dan verification e-certificate menggunakan algoritma RSA. Sistem memiliki dua proses utama, yaitu proses penerbitan sertifikat dan proses verification sertifikat.

Pada proses penerbitan, pengguna memasukkan file sertifikat berupa PDF atau PNG dan menentukan tanggal berlaku sertifikat. Sistem kemudian mengambil representasi isi dokumen dan menghasilkan hash menggunakan SHA-256. Hash tersebut digabungkan dengan tanggal berlaku menjadi payload yang kemudian diproses menggunakan private key untuk menghasilkan RSA token. Token tersebut disimpan pada metadata file sertifikat.

Pada proses verification, pengguna memasukkan sertifikat yang ingin diperiksa. Sistem mengambil RSA token dari metadata dan melakukan verification menggunakan public key. Hasil verification digunakan untuk mendapatkan kembali payload dan hash yang sebelumnya digunakan saat signing. Sistem kemudian menghitung kembali hash dari sertifikat yang diperiksa dan membandingkannya dengan hash dari payload. Tanggal berlaku juga diperiksa untuk menentukan apakah sertifikat masih dalam

masa berlaku. Secara umum, alur sistem dapat digambarkan sebagai berikut:

Penerbitan:

File Sertifikat → SHA-256 → Payload → RSA Signing → RSA Token → Metadata Sertifikat.

Verification:

Sertifikat → Ambil RSA Token → RSA Verification → Payload → Bandingkan Hash + Cek Expiry → Hasil Verification.

3.2. Proses Key Generation

Proses key generation pada sistem ini terdapat pada file `rsa_core.py`. Proses dimulai dengan memilih dua bilangan prima, yaitu p dan q , yang kemudian digunakan untuk menghitung nilai n dan $\phi(n)$. Setelah itu, program menentukan nilai e yang memenuhi syarat RSA dan mencari nilai d yang sesuai dengan nilai e dan $\phi(n)$.

Untuk menjalankan proses tersebut, program memiliki beberapa fungsi, yaitu `is_prime()` untuk memeriksa bilangan prima, `gcd()` untuk menghitung faktor persekutuan terbesar, `find_d()` untuk mencari nilai d , dan `generate_keypair()` untuk menghasilkan pasangan public key dan private key.

Hasil dari proses ini adalah public key yang terdiri dari nilai e dan n , serta private key yang terdiri dari nilai d dan n . Pada tampilan aplikasi, setiap tahapan pembangkitan kunci ditampilkan secara berurutan sehingga pengguna dapat melihat proses perhitungan hingga kedua kunci terbentuk.

1 Pilih dua bilangan prima $p \neq q$ **rahasia**

$$p = 10079$$



$$q = 10091$$

2 Modulus $n = p \times q$ **publik**

$$n = 101707189 \quad (27 \text{ bit})$$

3 Totient Euler $\Phi(n) = (p-1)(q-1)$ **rahasia**

$$\Phi(n) = 10078 \times 10090 = 101687020$$

4 Eksponen publik e , syarat $\text{PBB}(e, \Phi(n)) = 1$ **publik**

$$e = 65537$$

$$101687020 = 1551 \times 65537 + 39133$$

$$65537 = 1 \times 39133 + 26404$$

$$39133 = 1 \times 26404 + 12729$$

```

26404 = 2×12729 + 946
12729 = 13×946 + 431
946 = 2×431 + 84
431 = 5×84 + 11
84 = 7×11 + 7
11 = 1×7 + 4
7 = 1×4 + 3
4 = 1×3 + 1
3 = 3×1 + 0
PBB = 1 ✓ relatif prima

```

5 Eksponen privat d dari $e \cdot d = 1 + k \cdot \Phi(n)$ rahasia

```

k = 47594
d = (1 + 47594 × 101687020) / 65537 = 73846713
cek:  $e \cdot d \bmod \Phi(n) = 1$ 

```

Nilai k besar, tabel percobaan tidak ditampilkan.

6 Pasangan kunci terbentuk

Kunci publik (e, n) publik

e = 65537

n = 101707189

Kunci privat (d, n) rahasia

d = 73846713

n = 101707189

Sidik jari kunci publik: 5E46 0383 15B0 7301

3.3. Implementasi Modular Exponentiation

Modular exponentiation merupakan operasi perpangkatan dengan modulus yang digunakan dalam proses RSA. Pada sistem ini, operasi tersebut diimplementasikan melalui fungsi `mod_pow()` pada file `rsa_core.py` dengan menggunakan metode Square-and-Multiply.

Metode ini digunakan untuk menghitung perpangkatan modular dengan lebih efisien, terutama ketika nilai eksponen cukup besar. Prosesnya dilakukan dengan memecah eksponen menjadi representasi biner, kemudian

melakukan operasi perkalian dan pemangkatan secara bertahap dengan tetap menghitung modulus pada setiap langkah.

Dalam implementasi RSA, fungsi `mod_pow()` digunakan pada proses signing untuk menghitung nilai signature menggunakan private key dan pada proses verification untuk memperoleh kembali nilai payload menggunakan public key. Dengan demikian, operasi inti RSA dilakukan melalui implementasi sendiri tanpa menggunakan fungsi RSA dari library kriptografi.

▼

Simulasi perhitungan per blok: $m_i \rightarrow c_i = m_i^d \bmod n$

Payload `hash|tanggal` dipecah menjadi blok byte, tiap blok dipangkatkan dengan kunci privat `d`. Pilih blok untuk melihat langkah square-and-multiply.

Blok ke-

1

Isi blok (byte → karakter): `5e4` · `m` = 3499316

$c = m^d \bmod n$

`c` = 3499316 ^ 73846713 mod 101707189

`c` = 68304987

Eksponen `d` dalam biner (27 bit): 10001100110110011110111001

Eksponen panjang: menampilkan 12 langkah pertama dan 6 terakhir dari 27 langkah.

i	bit eksponen	basis ^(2ⁱ) mod n	hasil sementara
0		1 3499316	3499316
1		0 73741012	3499316
2		0 29387830	3499316
3		1 92566369	89847002
4		1 100365120	14228403
5		1 16590760	74659816
6		0 100571230	74659816
7		1 43742838	56894159
8		1 18928846	65984743
9		1 5979286	37829399
10		1 55114083	49305510
11		1 54811005	53808214
21		1 94683016	79645758
22		1 35292517	90456189
23		0 89283284	90456189
24		0 42742900	90456189
25		0 45365034	90456189
26		1 25454209	68304987

`c_1` = 68304987 ✓ sama dengan token

3.4. Proses Signing

Proses signing pada sistem ini dilakukan pada file `cert_service.py` dengan memanfaatkan fungsi RSA yang terdapat pada `rsa_core.py`. Sebelum proses signing dilakukan, sistem mengambil hash dari isi sertifikat dan menggabungkannya dengan tanggal berlaku sertifikat menjadi sebuah payload.

Payload tersebut kemudian diproses menggunakan private key untuk menghasilkan signature. Karena payload dapat terdiri dari beberapa blok, proses signing dilakukan pada setiap blok dengan operasi perpangkatan modular. Hasil dari setiap blok selanjutnya digunakan untuk membentuk RSA token.

RSA token yang dihasilkan menjadi tanda tangan digital untuk payload tersebut dan akan digunakan pada proses verification. Dengan cara ini, sistem dapat memeriksa apakah hash dari sertifikat yang diterima masih sesuai dengan hash yang ditandatangani saat sertifikat diterbitkan.



Sertifikat berhasil ditandatangani

Segel RSA disuntikkan ke metadata berkas. Tampilan visual tidak berubah. Berlaku s.d. 2027-10-09.

Jumlah blok	Ukuran chunk (byte)	Panjang token (karakter)	Format berkas
26	3	232	PDF

Hash visual dokumen (SHA-256)

5e415c26e24698f65d253c3c69ece216f409fd07530f84b05ca81f2e544e52ee



↓ Unduh dokumen bertanda tangan (PDF)

Sebelum (polos)



Sesudah (bertanda tangan, terlihat sama)



3.5. Pembentukan RSA Token

Setelah proses signing pada setiap blok selesai, hasilnya disusun menjadi sebuah RSA token. Token ini berisi hasil operasi RSA dari payload yang sebelumnya telah dibagi menjadi beberapa blok. Setiap blok diproses menggunakan private key, kemudian hasilnya digabungkan agar dapat disimpan dan digunakan kembali saat verification.

Pada sistem ini, RSA token disimpan dalam bentuk teks yang memuat hasil signing dari setiap blok. Token tersebut kemudian ditempatkan pada metadata file sertifikat dengan nama RSA_Token. Dengan cara ini, sistem dapat mengambil token secara langsung dari file saat proses verification dilakukan.

Panjang token bergantung pada jumlah blok payload yang diproses dan format penyimpanan yang digunakan. Token inilah yang menjadi bagian penting dalam proses verification karena digunakan untuk memperoleh kembali payload yang ditandatangani.

Seluruh blok & token (/RSA_Token)			
blok	m_i	karakter	c_i
1	3499316	5e4	68304987
2	3224931	15c	15159855
3	3290725	26e	5036238
4	3290166	246	75463679
5	3749990	98f	68384816
6	3552612	65d	13257612
7	3290419	253	27936296
8	6501219	c3c	67798826
9	3553637	69e	21707368
10	6513970	ce2	93916953
11	3225190	16f	36910057
12	3420217	409	18633040
13	6710320	fd0	21715287
14	3618099	753	45044573
15	3171896	0f8	91384401
16	3433008	4b0	95359025
17	3498849	5ca	6527332
18	3682662	81f	78462414
19	3302709	2e5	64133385
20	3421285	44e	52310276

21	3486309	52e	62641590
22	6650930	e 2	98937911
23	3158583	027	40355147
24	2961712	-10	33760574
25	2961465	-09	41187818
26	197379	...	5048725

3:68304987 15159855 5036238 75463679 68384816 13257612 27936296 67798826 21707368 93916953 36910057 18633040
21715287 45044573 91384401 95359025 6527332 78462414 64133385 52310276 62641590 98937911 40355147 33760574 41187818
5048725

3.6. Penyimpanan Token pada Metadata

Setelah RSA token terbentuk, sistem menyimpan token tersebut pada metadata file sertifikat. Penyimpanan dilakukan sesuai dengan format file yang digunakan, yaitu PDF atau PNG.

Pada file PDF, token disimpan menggunakan metadata dengan nama RSA_Token. Sementara itu, pada file PNG, token disimpan sebagai teks metadata. Dengan cara ini, token dapat dibaca kembali oleh sistem saat sertifikat akan diverifikasi.

Penyimpanan token pada metadata dilakukan agar informasi tanda tangan dapat disertakan dalam file sertifikat tanpa perlu menampilkannya pada bagian visual dokumen. Setelah proses penyimpanan selesai, sistem menghasilkan file sertifikat bertanda tangan yang dapat diunduh dan digunakan untuk proses verification.

```
Creator:      Microsoft® PowerPoint® 2016
Producer:     iLovePDF
CreationDate:  Fri Aug 28 08:31:29 2026 UTC
ModDate:      Fri Aug 28 08:31:33 2026 UTC

Custom Metadata: no
Metadata Stream: no
Tagged:       no
UserProperties: no
Suspects:     no
```

```
Producer:     pypdf
Custom Metadata: yes
Metadata Stream: no
Tagged:       no
UserProperties: no
Suspects:     no
Form:         none
JavaScript:   no
Pages:       44
```

```
PS C:\Users\Loven> d:
PS D:\> cd .\Akademik\
PS D:\Akademik> python cek_metadata.py
Masukkan path PDF: C:\Users\Loven\Downloads\pertemuan_4_lengkap-ok_compressed_signed.pdf

=== METADATA PDF ===
/Producer: pypdf
/RSA_Token: 3:68304987 15159855 5036238 75463679 68384816 13257612 27936296 67798826 21707368 93916953 36910057 18633040
21715287 45044573 91384401 95359025 6527332 78462414 64133385 52310276 62641590 98937911 40355147 33760574 41187818 504
8725

=== PEMERIKSAAN RSA TOKEN ===
RSA_Token ditemukan!
Isi token:
3:68304987 15159855 5036238 75463679 68384816 13257612 27936296 67798826 21707368 93916953 36910057 18633040 21715287 45
044573 91384401 95359025 6527332 78462414 64133385 52310276 62641590 98937911 40355147 33760574 41187818 5048725
PS D:\Akademik>
```

3.7. Proses Verification

Proses verification dilakukan untuk memeriksa tanda tangan digital yang terdapat pada sertifikat. Pada sistem ini, proses dimulai dengan mengambil RSA token dari metadata file yang diunggah. Token tersebut

kemudian diproses menggunakan public key untuk mendapatkan kembali payload yang sebelumnya ditandatangani.

Setiap blok pada token dihitung menggunakan operasi perpangkatan modular dengan rumus:

$$m_i = c_i^e \bmod n$$

Hasil dari setiap blok kemudian digabungkan untuk mendapatkan payload. Setelah itu, sistem mengambil hash dan tanggal berlaku yang terdapat pada payload. Hash tersebut dibandingkan dengan hasil perhitungan hash dari sertifikat yang sedang diperiksa.

Jika signature dapat diproses dengan benar dan hash dokumen sesuai, sistem akan melanjutkan pemeriksaan tanggal berlaku sertifikat. Hasil akhirnya ditentukan berdasarkan kecocokan hash, keberhasilan verification, dan status masa berlaku sertifikat.

▼ Simulasi perhitungan per blok: $c_i \rightarrow m_i = c_i^e \bmod n$

Tiap blok segel dipangkatkan dengan kunci publik e untuk memulihkan payload `hash|tanggal`.

blok	c_i	$m_i = c_i^e \bmod n$	karakter
1	68304987	3499316	5e4
2	15159855	3224931	15c
3	5036238	3290725	26e
4	75463679	3290166	246
5	68384816	3749990	98f
6	13257612	3552612	65d
7	27936296	3290419	253
8	67798826	6501219	c3c
9	21707368	3553637	69e
10	93916953	6513970	ce2
			-

Lihat langkah blok ke-

1

$$m = c^e \bmod n$$

```
m = 68304987 ^ 65537 mod 101707189
m = 3499316
```

Eksponen e dalam biner (17 bit): 10000000000000001

i	bit eksponen	basis^(2^i) mod n	hasil sementara
0	1	68304987	68304987
1	0	85092549	68304987
2	0	34570597	68304987
3	0	87977614	68304987
4	0	75096506	68304987
5	0	79474731	68304987
6	0	99431358	68304987
7	0	69847925	68304987
8	0	78077379	68304987
9	0	29849605	68304987

Payload pulihan: 5e415c26e24698f65d253c3c69ece216f409fd07530f84b05ca81f2e544e52ee|2027-10-09

3.8. Pemeriksaan Expiry

Setelah payload berhasil diperoleh melalui proses verification, sistem mengambil hash dokumen dan tanggal berlaku yang terdapat di dalam payload. Hash tersebut kemudian dibandingkan dengan hash yang dihitung kembali dari dokumen yang sedang diperiksa.

Jika kedua hash sama, berarti isi dokumen yang diperiksa masih sesuai dengan isi dokumen saat proses signing dilakukan. Sebaliknya, jika hash berbeda, sistem dapat mendeteksi bahwa dokumen telah mengalami perubahan.

Selain membandingkan hash, sistem juga memeriksa tanggal berlaku yang terdapat pada payload. Jika tanggal berlaku sudah terlewati, sertifikat akan ditandai sebagai EXPIRED. Jika hash sesuai dan masa berlaku masih aktif, sistem menampilkan status VERIFIED. Sementara itu, jika dokumen tidak memiliki RSA token, sistem menampilkan status UNSIGNED.

UNSigned

Dokumen tidak memiliki segel tanda tangan digital.

Daftar pemeriksaan

- ✗ Segel RSA (/RSA-Token) ditemukan di metadata
- Tanda tangan dapat dipulihkan dengan kunci publik & format payload valid
- Hash visual dokumen yang dihitung ulang sama dengan hash di dalam segel
- Masa berlaku belum lewat

Pratinjau setelah verifikasi

TAMPERED

Konten visual dokumen telah dimodifikasi setelah ditandatangani.

Daftar pemeriksaan

- ✓ Segel RSA (/RSA-Token) ditemukan di metadata
- ✓ Tanda tangan dapat dipulihkan dengan kunci publik & format payload valid
- ✗ Hash visual dokumen yang dihitung ulang sama dengan hash di dalam segel
- Masa berlaku belum lewat

Perbandingan hash

Hash di dalam segel (dipulihkan dengan kunci publik)

```
9a12ac797c589b043793f51beedbf5b586f179002b811da3efc27ce4b360b471
```

Hash dihitung ulang dari berkas yang diunggah

```
5407e289e994fe4071c4374f2a7eb9a858ca1863cb1274b517c84077c472e1c4
```

Berbeda: isi visual dokumen berubah setelah ditandatangani.



EXPIRED

Tanda tangan otentik tetapi masa berlaku dokumen telah habis sejak 2022-01-09.

Daftar pemeriksaan

- ✓ Segel RSA (/RSA_Token) ditemukan di metadata
- ✓ Tanda tangan dapat dipulihkan dengan kunci publik & format payload valid
- ✓ Hash visual dokumen yang dihitung ulang sama dengan hash di dalam segel
- ⚠ Masa berlaku belum lewat

Hash dokumen (cocok dengan segel)

5e415c26e24698f65d253c3c69ece216f409fd07530f84b05ca81f2e544e52ee

Hash di dalam segel = hash hasil hitung ulang.

3.9. Antarmuka Sistem

Pada bagian penerbitan sertifikat, pengguna dapat memasukkan dokumen dalam format PDF atau PNG dan menentukan tanggal berlaku sertifikat. Sistem kemudian melakukan proses signing menggunakan private key dan menyimpan RSA token pada metadata dokumen. Setelah proses selesai, sertifikat yang telah ditandatangani dapat diunduh oleh pengguna.

ditampilkan dalam bentuk status agar pengguna dapat memahami kondisi sertifikat.

Sistem menyediakan empat status utama, yaitu VERIFIED, TAMPERED, EXPIRED, dan UNSIGNED. Status VERIFIED menunjukkan bahwa tanda tangan dapat diverifikasi, hash dokumen sesuai dengan hash dalam payload, dan masa berlaku sertifikat masih aktif. Status TAMPERED menunjukkan bahwa hash visual dokumen berbeda dari hash yang ditandatangani. Status EXPIRED menunjukkan bahwa tanda tangan valid dan hash dokumen sesuai, tetapi masa berlaku sertifikat telah berakhir. Sementara itu, status UNSIGNED menunjukkan bahwa sistem tidak menemukan RSA token pada metadata dokumen.

Selain menampilkan status, antarmuka verifikasi juga menyediakan daftar pemeriksaan yang menunjukkan hasil validasi token, pemulihan payload, kecocokan hash, dan masa berlaku. Pada kondisi tertentu, sistem menampilkan perbandingan hash untuk membantu pengguna memahami penyebab hasil verifikasi. Dengan demikian, antarmuka tidak hanya menampilkan hasil akhir, tetapi juga membantu pengguna memahami tahapan pemeriksaan sertifikat.

The screenshot displays the 'Sistem Pengesahan E-Sertifikat RSA' web application. The interface is divided into a left sidebar and a main content area. The sidebar, titled 'RSA E-Certificate', includes a user profile for 'Kelompok 17 - Kriptografi (A)' with an active role, a public key '5E46 0383 15B0 7301', and a note about key size. The main content area features a progress bar with three steps: 'Bangkitkan kunci' (active), 'Terbitkan & tandatangani', and 'Verifikasi dokumen'. Below this is a navigation menu with 'Kunci Otoritas', 'Penerbitan', 'Verifikasi', and 'Alur & Lab Rumus'. The 'Pembangkitan Kunci Otoritas' section is active, showing a step-by-step process: 1. Selecting two prime numbers p=10079 and q=10091 (labeled 'rahasia'). 2. Calculating the modulus n=p*q=101707189 (27 bits, labeled 'publik'). 3. Calculating the totient Euler phi(n)=(p-1)(q-1) (labeled 'rahasia').

4. Analisis Hasil

4.1. Analisis Verifikasi Integritas

Verifikasi integritas pada sistem e-certificate dilakukan dengan membandingkan hash visual dokumen yang diperoleh saat proses verifikasi dengan hash yang terdapat pada payload hasil verifikasi RSA. Sebelum perbandingan dilakukan, sistem memulihkan payload dari RSA token menggunakan public key.

Berdasarkan mekanisme tersebut, sistem dapat mendeteksi perubahan pada representasi visual sertifikat setelah proses signing. Apabila kedua hash sesuai dan masa berlaku sertifikat masih aktif, sistem menampilkan status VERIFIED. Sebaliknya, apabila hash berbeda, sistem menampilkan status TAMPERED. Pemeriksaan tanggal berlaku dilakukan untuk menentukan apakah sertifikat masih berada dalam masa berlaku.

4.2. Analisis Hasil Pengujian

Pengujian dilakukan untuk mengetahui kemampuan sistem dalam menangani beberapa kondisi sertifikat. Hasil pengujian menunjukkan bahwa sistem menghasilkan status yang berbeda sesuai dengan kondisi dokumen yang diperiksa.

Kondisi Pengujian	Hasil yang Diharapkan	Status
Sertifikat bertanda tangan, hash sesuai, dan belum kedaluwarsa	Sertifikat valid	VERIFIED
Sertifikat mengalami perubahan visual setelah signing	Perubahan terdeteksi	TAMPERED
Sertifikat bertanda tangan dan hash sesuai, tetapi masa berlaku berakhir	Sertifikat kadaluarsa	EXPIRED
Sertifikat tidak memiliki RSA token	Tanda tangan tidak ditemukan	UNSIGNED

Hasil tersebut menunjukkan bahwa sistem dapat membedakan sertifikat berdasarkan kecocokan hash, keberadaan RSA token, dan masa berlaku. Status TAMPERED menunjukkan adanya ketidaksesuaian antara hash dokumen yang diperiksa dan hash yang ditandatangani. Status EXPIRED menunjukkan bahwa masa berlaku sertifikat telah berakhir, sedangkan status UNSIGNED menunjukkan bahwa sistem tidak menemukan RSA token pada metadata dokumen.

4.3. Keterbatasan Sistem

Sistem yang dikembangkan masih memiliki beberapa keterbatasan. Pertama, implementasi RSA dibuat secara manual untuk keperluan pembelajaran dan demonstrasi, sehingga belum ditujukan untuk penggunaan produksi. Kedua, pemeriksaan integritas dokumen bergantung pada representasi visual yang digunakan untuk menghitung hash. Perubahan yang tidak memengaruhi tampilan visual, seperti perubahan metadata selain RSA token, tidak selalu terdeteksi melalui pemeriksaan tersebut.

Selain itu, keamanan sistem juga bergantung pada kerahasiaan private key dan keaslian public key yang digunakan untuk verifikasi. Apabila public key yang digunakan tidak benar-benar berasal dari penerbit sertifikat, hasil verifikasi tidak dapat dijadikan bukti keaslian penerbit. Oleh karena itu, sistem ini lebih sesuai digunakan sebagai implementasi pembelajaran mengenai RSA, digital signature, dan pemeriksaan integritas e-certificate.

5. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, sistem e-certificate berhasil dikembangkan dengan menerapkan algoritma RSA secara manual untuk proses key generation, signing, dan verification. Sistem menggunakan SHA-256 untuk menghasilkan hash visual dokumen, kemudian menggabungkannya dengan tanggal berlaku sertifikat sebagai payload yang ditandatangani menggunakan private key.

RSA token disimpan pada metadata file sertifikat sehingga dapat digunakan kembali saat proses verifikasi tanpa mengubah tampilan visual dokumen. Sistem juga dapat memeriksa kecocokan hash dan masa berlaku sertifikat serta menampilkan status VERIFIED, TAMPERED, EXPIRED, atau UNSIGNED sesuai dengan kondisi yang diperiksa.

Hasil pengujian menunjukkan bahwa kombinasi RSA, SHA-256, dan pemeriksaan tanggal berlaku dapat digunakan untuk mendemonstrasikan proses verifikasi tanda tangan digital serta mendeteksi perubahan visual pada e-certificate. Namun, sistem masih memiliki keterbatasan, terutama pada aspek keamanan implementasi dan pemeriksaan perubahan yang tidak memengaruhi tampilan visual dokumen. Oleh karena itu, sistem ini lebih sesuai digunakan sebagai sarana pembelajaran dan demonstrasi penerapan digital signature pada e-certificate.

6. Lampiran

6.1. Screenshot bagian app.py

```
C:\Users\> Ioven > Downloads > rsa-eertificate-system-main > app.py > ...
14
15 import html
16 import io
17 from datetime import datetime, timedelta, timezone
18
19 import streamlit as st
20 from PIL import Image, ImageDraw, ImageFont
21 from pypdf import PdfReader
22
23 import rsa_core
24 from cert_service import (
25     compute_sha256,
26     extract_visual_bytes,
27     issue_certificate_file,
28     verify_certificate_file,
29 )
30 from rsa_core import generate_keypair
31
32 st.set_page_config(page_title="E-Certificate RSA System", page_icon="🔒", layout="wide")
33
34 MAX_PRIME_INPUT = 10**13 # batas aman uji prima trial division (backend)
35 STATUS_STYLE = {
36     "VERIFIED": ("s-verified", "✓", "Dokumen sah"),
37     "TAMPERED": ("s-tampered", "✗", "Dokumen dimanipulasi / segel tidak valid"),
38     "EXPIRED": ("s-expired", "⌚", "Dokumen otentik, tetapi masa berlaku habis"),
39     "UNSIGNED": ("s-unsigned", "🔒", "Dokumen tidak memiliki segel"),
40 }
41
42 # -----
43 # CSS
44 # -----
45 CSS = """
46 <style>
47 .block-container{padding-top:2.2rem;max-width:1200px}
48 .hero hi{margin:0 0 .15rem 0;font-size:2rem}
49 """
```

6.2. Screenshot bagian cert_service.py

```
C:\Users\> Ioven > Downloads > rsa-eertificate-system-main > cert_service.py > extract_visual_bytes
7 import hashlib
8 import io
9 from datetime import datetime, timezone
10 from PIL import Image
11 from PIL.PngImagePlugin import PngInfo
12 from pypdf import PdfReader, PdfWriter
13 import fitz # PyMuPDF: render PDF pages to their visible appearance
14 import rsa_core
15
16
17 def compute_sha256(data_bytes: bytes) -> str:
18     """Menghitung ringkasan SHA-256 dari aliran byte."""
19     return hashlib.sha256(data_bytes).hexdigest()
20
21
22 def extract_visual_bytes(file_bytes: bytes, file_type: str) -> bytes:
23     """Return a deterministic representation of the document's visible content.
24
25     - PDF: render every page with fixed settings and hash the rendered RGB pixels.
26     Page annotations are rendered too, so visible markup/images are included.
27     PDF metadata is not part of this representation, avoiding a token/hash loop.
28     - PNG: convert to RGBA and return raw pixel bytes.
29     """
30     ext = file_type.lower().lstrip(".")
31
32     if ext == "pdf":
33         try:
34             document = fitz.open(stream=file_bytes, filetype="pdf")
35         except Exception as exc:
36             raise ValueError("Dokumen PDF tidak dapat dibaca atau dirender.") from exc
37
38         if document.needs_pass:
39             document.close()
40             raise ValueError("PDF terenkripsi dengan kata sandi tidak didukung.")
41         if len(document) == 0:
```

6.3. Screenshot bagian requirement.txt

```
requirements.txt
File Edit View H1 [icons] B I S [icons] [icons] [icons]
streamlit[pdf]>=1.50.0
pypdf>=5.0.0
Pillow>=10.4.0
pytest>=8.3.0
PyMuPDF>=1.24.0
```

6.4. Screenshot bagian rsa_core.py

```
C:\Users\Loven > Downloads > rsa-ecertificate-system-main > rsa_core.py > ...
6 def is_prime(num: int) -> bool:
7     """Uji bilangan prima manual menggunakan metode Trial Division."""
8     if num < 2:
9         return False
10    if num in (2, 3):
11        return True
12    if num % 2 == 0 or num % 3 == 0:
13        return False
14    i = 5
15    while i * i <= num:
16        if num % i == 0 or num % (i + 2) == 0:
17            return False
18        i += 6
19    return True
20
21
22 def gcd(a: int, b: int) -> int:
23     """Kalkulasi Greatest Common Divisor menggunakan Algoritma Euclidean."""
24     while b != 0:
25         a, b = b, a % b
26     return a
27
28
29 def extended_gcd(a: int, b: int) -> tuple[int, int, int]:
30     """
31     Algoritma Extended Euclidean.
32     Menghasilkan (gcd, x, y) sedemikian rupa sehingga a*x + b*y = gcd(a, b).
33     """
34     if a == 0:
35         return b, 0, 1
36     g, x1, y1 = extended_gcd(b % a, a)
37     x = y1 - (b // a) * x1
38     y = x1
39     return g, x, y
```

6.5. Screenshot bagian cek_metadata.py

```
C:\Users\Loven > Downloads > rsa-ecertificate-system-main > cek_metadata.py
1
2 from pypdf import PdfReader
3
4 file_path = input("Masukkan path PDF: ")
5 reader = PdfReader(file_path)
6
7 metadata = reader.metadata
8
9 print("\n=== METADATA PDF ===")
10 if metadata:
11     for key, value in metadata.items():
12         print(f"{key}: {value}")
13 else:
14     print("Tidak ada metadata.")
15
16 print("\n=== PEMERIKSAAN RSA TOKEN ===")
17 if metadata and "/RSA_Token" in metadata:
18     token = metadata["/RSA_Token"]
19     print("RSA_Token ditemukan!")
20     print(f"Isi token: {token}")
21 else:
22     print("RSA Token tidak ditemukan pada metadata dokumen.")
23
```

6.6. Link github repository

Untuk Kode lengkap dapat dilihat pada link github dibawah :

[Link Github](#)